Bank Customer Churn Prediction

Flask web app with a neural network (scikit-learn MLP), deployed with Docker on AWS

Author: Simon V | GitHub: SimonAdvika

1. Project Title

Bank Customer Churn Prediction Using a Neural Network (MLP), Flask and Docker on AWS

The project predicts whether a bank customer is likely to leave the bank (churn) or stay.

```
Customer information -> Preprocessing -> MLP neural network -> Churn
probability -> Likely to CHURN / Likely to STAY
```

2. Abstract

This project builds an end-to-end customer churn prediction system for a bank. A neural network (scikit-learn MLPClassifier) is trained on historical customer records using these inputs: credit score, geography, gender, age, tenure, account balance, number of products, credit card status, active member status and estimated salary.

A Flask web application loads the trained model, scales the user's input with the saved scaler, and returns a churn probability and a prediction: **Likely to CHURN** or **Likely to STAY**. The application is packaged as a Docker image, stored in Amazon ECR and run on AWS with ECS Express Mode, so it is available through a public URL.

The purpose is to help a bank identify customers at risk of leaving so that it can take retention actions.

3. Introduction

Banks have a large number of customers. Some keep using the services, while others close their account, become inactive or move to another bank. This is called **customer churn**.

Predicting churn is useful because keeping an existing customer is often more efficient than continuously acquiring new ones. This project uses data science and machine learning to find patterns in historical customer data that are associated with churn.

4. Problem Statement

How can we predict whether a bank customer is likely to leave the bank using historical customer data?

Example input:

```
Age = 45, Credit Score = 650, Balance = 120000, Products = 2, Active Member = Yes
```

The model processes these values and returns a probability, then a prediction: LIKELY TO STAY or LIKELY TO CHURN.

5. Objectives

1. **Data analysis:** understand customer information and identify patterns.
2. **Data preprocessing:** remove unneeded columns, handle missing values, encode categories and scale features.
3. **Model training:** train a neural network classifier on historical data.
4. **Model evaluation:** measure performance with accuracy, precision, recall, F1-score and the confusion matrix.
5. **Prediction:** predict churn probability for a new customer through a web form.
6. **Deployment:** containerize the app with Docker and run it online on AWS.

6. Dataset

The training script reads `European_Bank.csv`, a bank customer dataset.

Column	Meaning
CreditScore	Customer credit score
Geography	France, Germany or Spain
Gender	Female or Male
Age	Customer age
Tenure	Years with the bank
Balance	Account balance

Column	Meaning
NumOfProducts	Number of bank products
HasCrCard	Credit card ownership (0/1)
IsActiveMember	Active member (0/1)
EstimatedSalary	Estimated salary
Exited	Target: 0 = stayed, 1 = churned

`train_model.py` looks for the target column automatically (Exited, Churn, churn, Customer_Churn or CustomerChurn). Check that your CSV column names match the table above.

7. Data Preprocessing

The training script (`train_model.py`) performs these steps in order:

1. **Load** the CSV with pandas and print its shape and first rows.
2. **Drop identifier columns** that carry no predictive meaning: RowNumber, CustomerId, CustomerID, Surname and Year (only those that exist).
3. **Check missing values** and remove rows that contain them.
4. **Separate features and target:** X holds customer information, y holds the Exited column.
5. **Clean the target:** text labels such as yes/no or churn/retained are mapped to 1/0, then converted to integers.
6. **Encode categorical columns** (see section 8).
7. **Save the feature column list** to `feature_columns.pkl` so the app uses the same column order.
8. **Split** the data (section 13) and **scale** it (section 9).

8. Categorical Data Encoding

Geography and Gender are text values, but the model needs numbers. This project uses **one-hot encoding** with `pd.get_dummies(..., drop_first=False)`.

```
Geography -> Geography_France, Geography_Germany, Geography_Spain
Gender    -> Gender_Female, Gender_Male
```

Each customer gets a 1 in the column of their category and 0 in the others. One-hot encoding avoids the false idea that Spain is bigger than France, which a simple 0/1/2 label encoding would suggest.

In `app.py`, the same encoding is applied to the single customer entered in the form, and `reindex(columns=feature_columns, fill_value=0)` makes sure the columns match the training data exactly.

9. Feature Scaling

Numerical values have very different ranges, for example Age 45, CreditScore 650 and Balance 125000. Neural networks train better when inputs are on a similar scale.

The project uses `StandardScaler`, which is **fitted only on the training data** and then applied to the test data. This prevents information from the test set leaking into training. The fitted scaler is saved as `churn_scaler.pkl`, and the web app uses the same scaler for every prediction.

10. Exploratory Data Analysis (EDA)

EDA helps us understand the dataset before training. Typical analyses for this project:

- **Churn distribution:** how many customers stayed and how many left.
- **Age vs churn:** whether churn varies across age groups.
- **Geography vs churn:** churn patterns by country.
- **Active membership vs churn:** whether inactive customers behave differently.
- **Balance and number of products vs churn.**

The EDA work is done in the notebook `Bank_Customer_Churn.ipynb`. Add your own charts and findings here.

11. Machine Learning Approach

Because the target has two outcomes (0 = stay, 1 = churn), this is a **binary classification** problem. Other algorithms such as Logistic Regression, Random Forest or SVM can be used as comparison baselines. The deployed model is a neural network.

12. Neural Network Model (scikit-learn MLP)

The deployed model is a **Multi-Layer Perceptron** from scikit-learn, **not TensorFlow/Keras**. This keeps the Docker image small and the deployment simple.

Setting	Value
Hidden layers	(64, 32, 16) neurons
Activation	ReLU
Optimizer	Adam, learning rate 0.001
Batch size	32
Regularization (alpha)	1e-4
Max iterations	200
Early stopping	Yes, 20% validation split, stops after 10 epochs without improvement
Random state	42

Flow:

```
Customer input -> One-hot encoding -> StandardScaler -> MLP -> predict_proba
-> churn probability
```

The output is a probability between 0 and 1. A probability of **0.50 or higher** is classified as churn.

13. Train/Test Split

- **80%** of the data is used for training and **20%** for testing.
- `random_state=42` makes the split repeatable.
- `stratify=y` keeps the same churn/stay ratio in both parts, which matters because churn data is usually imbalanced.

14. Model Evaluation

The script prints test accuracy, a classification report and the confusion matrix.

- **Accuracy:** percentage of all predictions that are correct. It is not enough on its own for imbalanced data.
- **Precision:** of the customers predicted to churn, how many actually churned.
- **Recall:** of all customers who actually churned, how many the model found.
- **F1-score:** the balance between precision and recall.

Confusion matrix:

	Predicted Stay	Predicted Churn
Actual Stay	True Negative	False Positive
Actual Churn	False Negative	True Positive

Results (fill in from your own output): Accuracy = ____ %, Precision (Churn) = ____, Recall (Churn) = ____, F1 (Churn) = ____.

Use the real numbers printed by `train_model.py`. Do not put estimated numbers in the final report.

15. Model Saving

The model is trained once and saved, so it does not need retraining each time someone opens the website. All three files are saved in the `models/` folder:

```
models/
  bank_churn_mlp.pkl      trained neural network
  churn_scaler.pkl        fitted StandardScaler
  feature_columns.pkl     expected feature names and order
```

These three files must come from the same training run. `app.py` checks at startup that the model, scaler and feature list all expect the same number of features, and stops with a clear message if they do not match. If a file is missing, it asks you to run `python train_model.py`.

16. Flask Web Application

`app.py` is a Flask application with two routes:

- `GET /` shows the web page (`templates/index.html`) with the input form.

- `POST /predict` receives the customer data as JSON and returns the result as JSON.

Input validation: the app rejects values outside sensible ranges and returns a clear error message.

Field	Allowed values
credit_score	300 to 900
geography	France, Germany or Spain
gender	Female or Male
age	18 to 100
tenure	0 to 50 years
balance, salary	0 to 1,000,000,000
products	1 to 10
has_card, active_member	0 or 1

Request flow:

```
User -> Web form -> POST /predict (JSON) -> Validation -> One-hot encoding ->
Scaler -> MLP model -> JSON response -> Result on page
```

17. Prediction Result

The response contains a success flag, the prediction text, a status (churn or stay) and the probability as a percentage. The page shows:

- **Customer is likely to STAY** (churn probability below 50%)
- **Customer is likely to CHURN** (churn probability 50% or higher)

For example: Churn Probability: 72%. This is a model score, not a guarantee about the customer.

18. Project Folder Structure

```
Bank Customer Churn/
  app.py
  train_model.py
  European_Bank.csv
  Bank_Customer_Churn.ipynb
  requirements.txt
  Dockerfile
  README.md
  .gitignore
  templates/
    index.html
  models/
    bank_churn_mlp.pkl
    churn_scaler.pkl
    feature_columns.pkl
```

- **app.py:** the Flask application.
- **train_model.py:** trains the model and saves the three files in `models/`.
- **Bank_Customer_Churn.ipynb:** EDA and experiments.
- **requirements.txt:** Python dependencies.
- **Dockerfile:** builds the container image.
- **templates/index.html:** the web page.

19. Requirements

Example `requirements.txt`:

```
flask
numpy
pandas
scikit-learn==X.X.X
gunicorn
```

Pin the exact scikit-learn version you trained with (check it with `pip show scikit-learn`). A different version inside the container can break loading of the saved pickle files. TensorFlow is **not** needed.

20. Run the Project Locally

Open PowerShell in the project folder:

```
cd "C:\UnifiedMentor\Bank Customer Churn"
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python train_model.py
python app.py
```

Then open http://127.0.0.1:5000 in your browser.

21. Docker

The Dockerfile uses `python:3.11-slim`, installs the requirements, copies the project (including `models/`) and exposes port 5000.

Recommended start command (production server):

```
CMD ["gunicorn", "-b", "0.0.0.0:5000", "app:app"]
```

Build and test locally:

```
docker build -t churn-app .
docker run -p 5000:5000 churn-app
```

22. Upload to GitHub

```
git init
git add .
git commit -m "Bank customer churn prediction - Flask + MLP + Docker"
git branch -M main
git remote add origin https://github.com/SimonAdvika/bank-customer-churn-
prediction.git
git push -u origin main
```

Create a `.gitignore` so that virtual environments, cache files, secrets and AWS keys are never committed. Never put access keys in the repository.

23. Deploy Online on AWS

The application is deployed with **Amazon ECR** and **Amazon ECS Express Mode** in the Asia Pacific (Sydney) region, `ap-southeast-2`.

1. **Create an ECR repository** named `churn-app`.
2. **Log in, build, tag and push** the Docker image:

```
aws ecr get-login-password --region ap-southeast-2 | docker login --username
AWS --password-stdin 006591011046.dkr.ecr.ap-southeast-2.amazonaws.com
docker build -t churn-app .
docker tag churn-app:latest 006591011046.dkr.ecr.ap-southeast-
2.amazonaws.com/churn-app:latest
docker push 006591011046.dkr.ecr.ap-southeast-2.amazonaws.com/churn-
app:latest
```

3. **Create an ECS Express Mode service** with the ECR image URI, container port 5000 and health check path `/`. Express Mode creates the cluster, Fargate tasks, load balancer and a public HTTPS URL automatically.
4. **Open the public URL** and test a prediction.

AWS App Runner no longer accepts new customers (since April 30, 2026), which is why ECS Express Mode is used.

Live demo: https://YOUR-ECS-URL-HERE

Complete deployment flow:

```
Python project -> Test locally -> Docker image -> Amazon ECR -> ECS Express
Mode -> Public website
```

Cost note: the load balancer is billed hourly even when nobody visits. Delete the service after the demo and redeploy from the same ECR image when needed.

24. Limitations

- Predictions depend on the quality and size of the training dataset.
- Historical patterns may not represent future customer behavior.
- The model can make incorrect predictions, and churn data is often imbalanced, which affects evaluation.

- The 0.50 probability threshold is a default and can be tuned to favor recall or precision.
- The app has no authentication, and real banking systems need strong security and privacy controls.
- The model should not be the only factor in real customer decisions.
- The live link is available only while the AWS service is running.

25. Future Enhancements

- **Explainable AI:** use SHAP to explain why a customer is predicted to churn.
- **Better dashboard:** churn percentage, customer statistics, interactive charts and geography analysis.
- **Model comparison and tuning:** compare against Random Forest or gradient boosting and tune the decision threshold.
- **Database:** connect the application to MySQL or PostgreSQL.
- **Authentication:** add login for authorized users.
- **CI/CD:** automatically build and deploy to AWS when code is pushed to GitHub.
- **Model monitoring and automatic retraining** with new customer data.

26. Conclusion

The Bank Customer Churn Prediction project demonstrates the practical use of data science, machine learning and cloud deployment on a real-world banking problem. The system takes customer information, applies the same preprocessing used in training, and uses a trained neural network to estimate whether a customer is likely to churn.

The project covers an end-to-end workflow: dataset analysis and model development, a Flask web application with input validation, Docker packaging, and online deployment on AWS using Amazon ECR and ECS Express Mode. Users can enter customer details through a simple web interface and receive a model-based churn probability.

Although it is mainly a data science and portfolio project, it provides a foundation for future improvements such as explainable AI, richer analytics, database integration, authentication, monitoring and automated retraining.